

# • Git, el Guardián del código •

## 1. ¿Qué es el control de versiones?

Imagina que estás escribiendo un documento muy importante. Lo guardas como `Tesis_V1.docx`. Haces cambios y lo guardas como `Tesis_V2_corregida.docx`. Luego como `Tesis_V3_ahora_si_final.docx` y finalmente como `Tesis_FINAL_FINAL_ESTA_SI.docx`. Es un caos. ¿Qué pasa si te das cuenta de que una idea que borraste en la versión 2 era en realidad muy buena? Encontrarla sería una pesadilla.

Ahora imagina que estás jugando un videojuego de aventura. Antes de entrar a la cueva del jefe final, ¿qué haces? ¡Guardas la partida! Si pierdes, no tienes que empezar todo el juego desde el principio, simplemente cargas ese punto de guardado y lo intentas de nuevo.

**Git es un sistema de control de versiones.** Es un software que te permite "guardar la partida" de tu código. Lleva un registro de cada cambio que haces, quién lo hizo y cuándo. Si algo sale mal, puedes "cargar una partida anterior" y volver a una versión de tu código que sí funcionaba.

## 2. Conceptos clave de Git

- **Repositorio (Repository / Repo):** Es la carpeta de tu proyecto que está siendo "vigilada" por Git. Contiene todos tus archivos y el historial completo de cambios.
- **Commit (Confirmación):** Es un "punto de guardado". Es una instantánea (snapshot) de tu proyecto en un momento específico. Cada commit tiene un mensaje único que describe los cambios que hiciste (ej: "Se agregó el botón de inicio de sesión").
- **Branch (Rama):** Imagina la línea de tiempo de tu proyecto como el tronco principal de un árbol. Si quieres experimentar con una nueva funcionalidad (por ejemplo, añadir un "modo oscuro") sin arriesgarte a romper lo que ya funciona, creas una **rama**. Esta es una línea de tiempo paralela donde puedes hacer cambios de forma segura.
- **Merge (Fusión):** Una vez que terminaste de desarrollar la nueva funcionalidad en tu rama y estás seguro de que funciona bien, puedes **fusionarla** con la rama principal. Git inteligentemente combina los cambios de ambas ramas.

### 3. Flujo de trabajo básico

El proceso de hacer un "commit" tiene dos pasos:

1. **Área de Preparación (Staging Area):** Antes de guardar la partida, tienes que elegir qué cambios quieres incluir en ese punto de guardado. Imagina que es una caja donde vas poniendo los archivos que ya están listos. El comando para esto es `git add <nombre_del_archivo>`.
2. **Hacer el Commit:** Una vez que tienes en la "caja" (Staging Area) todo lo que quieres guardar, confirmas esos cambios con un commit. Es en este momento cuando se crea el punto de guardado permanente en el historial. El comando es `git commit -m "Un mensaje descriptivo"`.

**Flujo:** Archivos Modificados → `git add` → Staging Area → `git commit` → Repositorio.

# • GitHub, la Red Social para Desarrolladores •

## 1. Diferencia entre Git y GitHub

Este punto es crucial y a menudo confunde a los principiantes.

- **Git:** Es la **herramienta**. Es el software que instalas en tu computadora para rastrear los cambios en tu código. Git no necesita internet para funcionar. Es el sistema de control de versiones en sí.
- **GitHub:** Es la **plataforma en la nube**. Es un servicio web (como Google Drive o Dropbox, pero para código) que aloja tus repositorios de Git.

**Analogía:** **Git** es como el programa Microsoft Word que tienes en tu PC para escribir novelas. **GitHub** es como Amazon, la plataforma donde puedes publicar y compartir tus novelas con el mundo. Otros "Amazons" para código son **GitLab** y **Bitbucket**.

## 2. ¿Para qué usamos GitHub?

- **Copia de Seguridad en la Nube:** Si tu computadora se daña, ¡tu código está a salvo en GitHub! Puedes clonar tu repositorio en cualquier otra máquina y seguir trabajando.
- **Portafolio Profesional:** ¡Esta es la clave para ustedes! Tu perfil de GitHub se convierte en tu CV como desarrollador. Los reclutadores no solo leen sobre tus habilidades, sino que pueden **ver tu código**, tus proyectos y tu actividad. Un perfil de GitHub activo es una prueba poderosa de tu pasión y conocimiento.
- **Colaboración:** GitHub es la herramienta por excelencia para trabajar en equipo. Varios desarrolladores pueden trabajar en el mismo proyecto, cada uno en su propia rama, y luego fusionar sus cambios de manera ordenada.

### • 3. Introducción a la colaboración: Pull Requests (PR) •

Cuando trabajas en un equipo (o quieres contribuir a un proyecto de código abierto), no puedes simplemente fusionar tus cambios en la rama principal. Tienes que **proponer** tus cambios.

Un **Pull Request** (Solicitud de Fusión) es exactamente eso: una solicitud formal que le haces al dueño del proyecto. Le dices: "Hola, he terminado mi trabajo en esta rama y creo que está listo. ¿Podrías por favor revisar mis cambios y, si estás de acuerdo, fusionarlos al proyecto principal?".

Esto permite que otros desarrolladores revisen tu código, dejen comentarios y se aseguren de que todo cumple con los estándares de calidad antes de integrarlo.

## • Instalación de Git •

1. Ve al sitio web oficial: <https://git-scm.com/downloads>.
2. Descarga la versión para tu sistema operativo.
3. **Instalación en Windows:** Ejecuta el instalador. Te presentará MUCHAS pantallas de configuración. Para empezar, simplemente haz clic en "Next" en todas las pantallas, dejando las opciones por defecto. Las opciones predeterminadas son perfectas para comenzar.
4. **Instalación en macOS:** Ejecuta el instalador .dmg y sigue los pasos. Si tienes Homebrew, puedes simplemente abrir la terminal y escribir `brew install git`.
5. **Verificación:** Abre la terminal (en Windows, abre "Git Bash" que se instaló con Git) y escribe `git --version`. Si te devuelve un número de versión, ¡Git está instalado correctamente!

## Cofiguración inicial de Git

Antes de usar Git, debes decirle quién eres. Abre la terminal y escribe estos dos comandos, reemplazando los datos con los tuyos.

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu_email@ejemplo.com"
```

- Crea una cuenta en GitHub: Ve a <https://github.com/> y regístrate. Es gratis.
- Crea una carpeta para tu proyecto: Abre la terminal. Vamos a crear una carpeta en el Escritorio.

```
cd Desktop
mkdir mi-primer-repo
cd mi-primer-repo
```